



Tutorial: proyecto Laravel con GitHub, agentes y MCP

Dirigido a: **Tenant admin / project owner que arranca un proyecto Laravel real**

Este tutorial monta, de cero y de punta a punta, un proyecto **Laravel** respaldado por un repositorio de **GitHub**, con un equipo de agentes **especializado en Laravel** (verás cómo: la plataforma no trae un equipo Laravel de serie, así que adoptaremos el equipo *Backend / API* built-in y lo especializaremos con una persona y una skill Laravel listas para copiar), y conectado por MCP a **Context7** (documentación de Laravel 11.x al día) y a **Atlassian** (Confluence + Jira). Al final tendrás un plan ejecutándose, el trabajo llegando como Pull Request a tu GitHub y la documentación fluyendo a Confluence.

El flujo es el mismo probado del manual 14: **requisitos** → **proyecto** → **equipo (+ especialización)** → **MCPs** → **tools** → **skills** → **primer plan** → **seguimiento** → **PR**. Donde Laravel difiere de CI4 (runtime de tests, comandos artisan, .env y APP_KEY, permisos de storage) este manual lo cubre con su paso propio. Sustituye tuorg/turepo y tuempresa.atlassian.net por los tuyos; los secretos van SIEMPRE en Vault.

Contenido

- 01** Requisitos previos (una sola vez)
- 02** Crear el proyecto Laravel
- 03** Adoptar el equipo Backend / API...
- 04** ...y especializarlo en Laravel (persona + prompt)
- 05** MCP 1/2 — Context7 (docs de Laravel al día)
- 06** MCP 2/2 — Atlassian (Confluence + Jira)
- 07** Asignar las tools MCP por mínimo privilegio
- 08** Skills Laravel: hábitos compartidos del equipo
- 09** El primer plan Laravel: qué pedir
- 10** Aprobar, ejecutar y seguir en vivo
- 11** Troubleshooting específico de Laravel

1 Requisitos previos (una sola vez)

Idénticos a cualquier proyecto con GitHub + Atlassian:

- **Proveedor LLM activo** (Plataforma → Proveedores LLM, botón «Probar» en verde). Cualquiera de los cuatro caminos del catálogo sirve, y puedes cambiarlo cuando quieras — el *vigía de credenciales* avisa si alguno caduca.
- **PAT de GitHub** con permiso `repo` sobre `tuorg/turepo` (push de ramas de plan + PRs automáticos).
- **API token de Atlassian** con escritura en tu espacio de Confluence y tu proyecto de Jira.
- **El repositorio GitHub** con la rama `main` inicializada. Consejo Laravel: si el repo ya contiene un esqueleto (`laravel new` commiteado), los agentes parten de él; si está vacío, la primera tarea del plan será crearlo — ambas rutas funcionan, pero dilo explícitamente en el plan (regla de tareas autocontenidas).

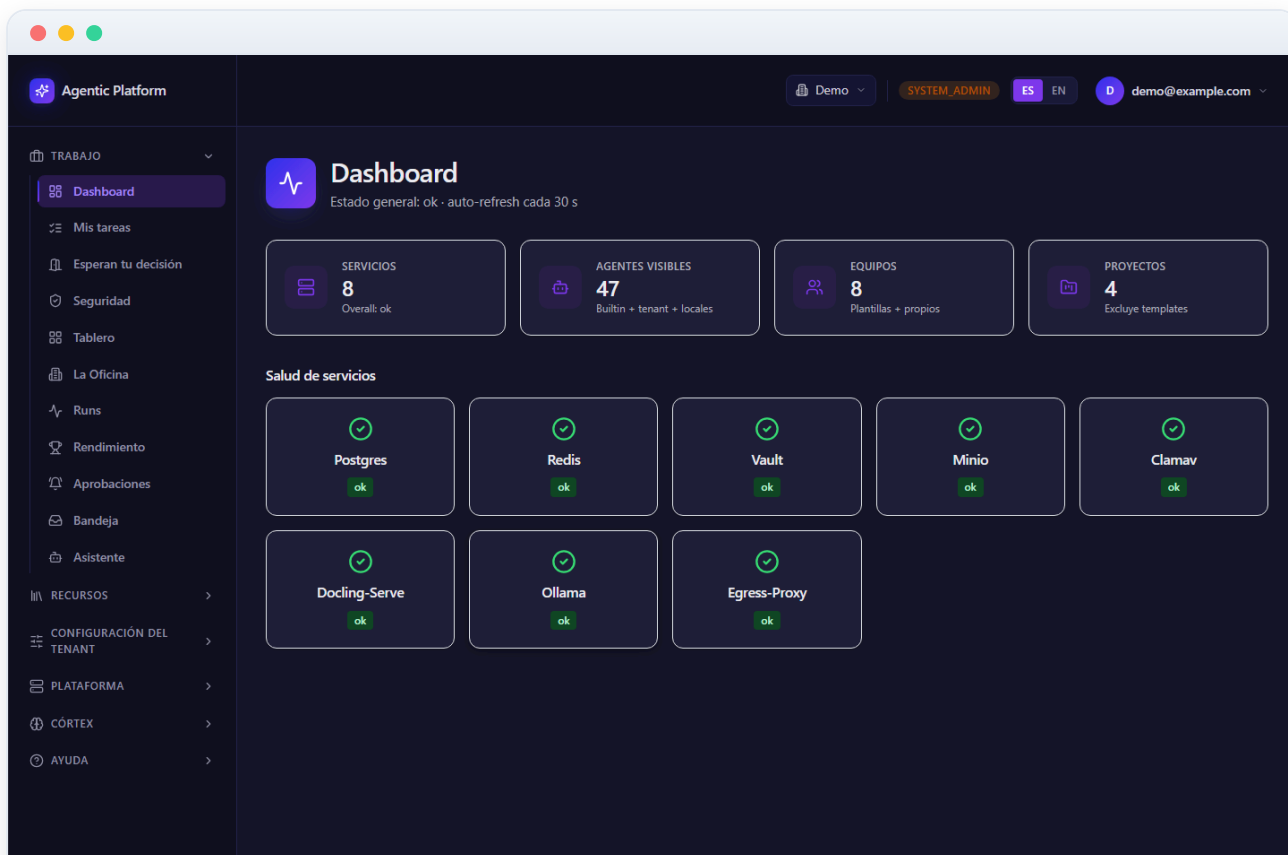


Figura 15.1 — Requisitos previos (una sola vez)

2 Crear el proyecto Laravel

En **Recursos** → **Proyectos** → «Nuevo proyecto»:

- **Nombre:** p. ej. `API Laravel`.
- **Stack:** `php`.
- **Runtime template:** `php-phpunit` si tu suite usa PHPUnit (el default de Laravel), o `php-pest` si usas **Pest** — ambos runtimes existen en el catálogo y traen las extensiones PHP necesarias (`intl` incluida). El runtime es el contenedor efímero donde los agentes ejecutan composer/artisan/tests vía `stack_exec`; los workers nunca ejecutan código de usuario.
- **Git remoto:** `https://github.com/tuorg/turepo.git`, rama `main`, PAT en Vault (ruta que indica el formulario).
- **Comandos permitidos** — la lista Laravel completa: `composer`, `php`, `vendor/bin/phpunit` (o `vendor/bin/pest`). Con `php` permitido, los agentes pueden ejecutar `php artisan ...` (migraciones, `make:*`, `key:generate`, `route:list`) — `artisan ES php`, no necesita entrada propia.
- **Dominios permitidos:** vacío por ahora; `mcp.context7.com` se añade en el paso de Context7.

Cada plan será una rama `plan/<id>-<slug>` en tu GitHub con commits trazados (`Plan-Id` / `Task-Id` / `Execution-Id`) y PR automático al completarse.

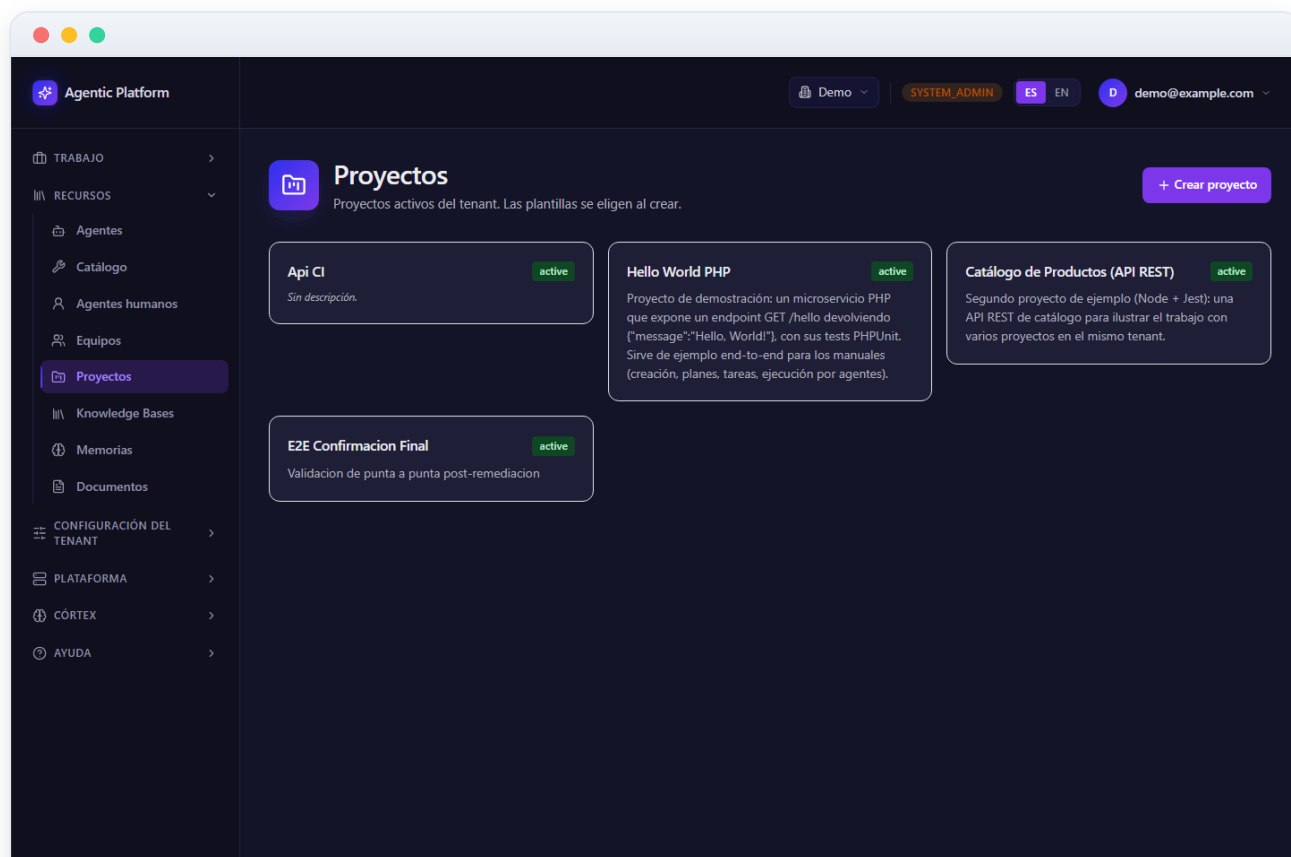


Figura 15.2 — Crear el proyecto Laravel

3 Adoptar el equipo Backend / API...

La plataforma trae 6 equipos built-in (CodeIgniter 4, Full-Stack Web, Backend / API, Research & Spec, DevOps & Platform, Data) — **no hay equipo Laravel de serie**, y no pasa nada: los equipos built-in genéricos + una especialización de persona/skill dan el mismo resultado. Elige según tu proyecto:

- **Equipo Backend / API** — para una API Laravel (nuestro caso): PM, arquitecto, backend devs, QA, reviewer, writer.
- **Equipo Full-Stack Web** — si tu Laravel lleva frontend Blade/Livewire/Inertia con entidad propia.

En **Recursos** → **Equipos**, pestaña built-in, pulsa «**Adoptar**» sobre *Backend / API* (ADR 0066: la adopción crea TU copia editable sin tocar el original) y asócialo al proyecto [API Laravel](#). La **asignación por rol** (ADR 0091) despachará cada tarea al agente de su rol.

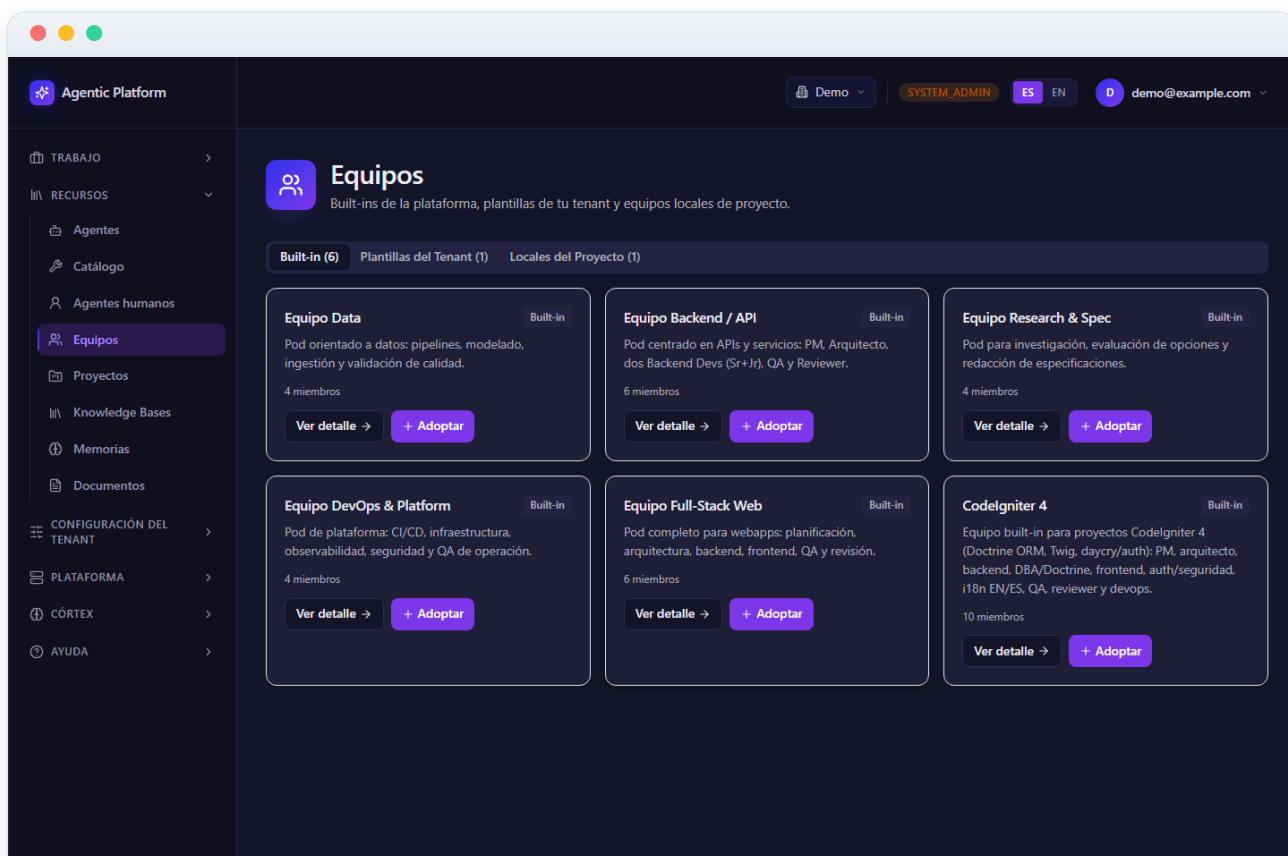


Figura 15.3 — Adoptar el equipo Backend / API...

...y especializarlo en Laravel (persona + prompt)

Ahora conviertes agentes genéricos en agentes Laravel. Abre la ficha de cada dev/arquitecto/QA de TU copia del equipo → sección **Persona** → **Editar**, y AÑADE al system prompt un bloque de especialización (listo para copiar, ajústalo a tu versión):

Especialización Laravel (11.x):

- Sigue las convenciones del framework: Eloquent para el acceso a datos (cuidado con N+1: usa `with()/loadMissing()`), `FormRequest` para la validación, `Policies` para autorización, y recursos API (`JsonResource`) para las respuestas.
- Estructura: código de dominio en `app/`, rutas en `routes/api.php`, migraciones SIEMPRE con `down()` funcional, seeders idempotentes.
- Genera artefactos con `artisan` (`php artisan make:model -mfs`, `make:controller --api`, `make:request`) en vez de crear ficheros a mano.
- Tests: Feature tests contra la API (`RefreshDatabase` con `sqlite` en memoria), `Unit` para lógica pura. Un endpoint sin test no está hecho.
- Config por `.env` + `config()`; JAMÁS leas `env()` fuera de `config/*`.

Hazlo al menos para el **Backend Dev** y el **Arquitecto** (los que escriben código); el QA agradece un bloque análogo orientado a Feature tests. La persona viaja como primer bloque del system prompt de TODOS sus runs — es la vía correcta para identidad permanente (los hábitos compartibles van en skills, dos pasos más abajo).

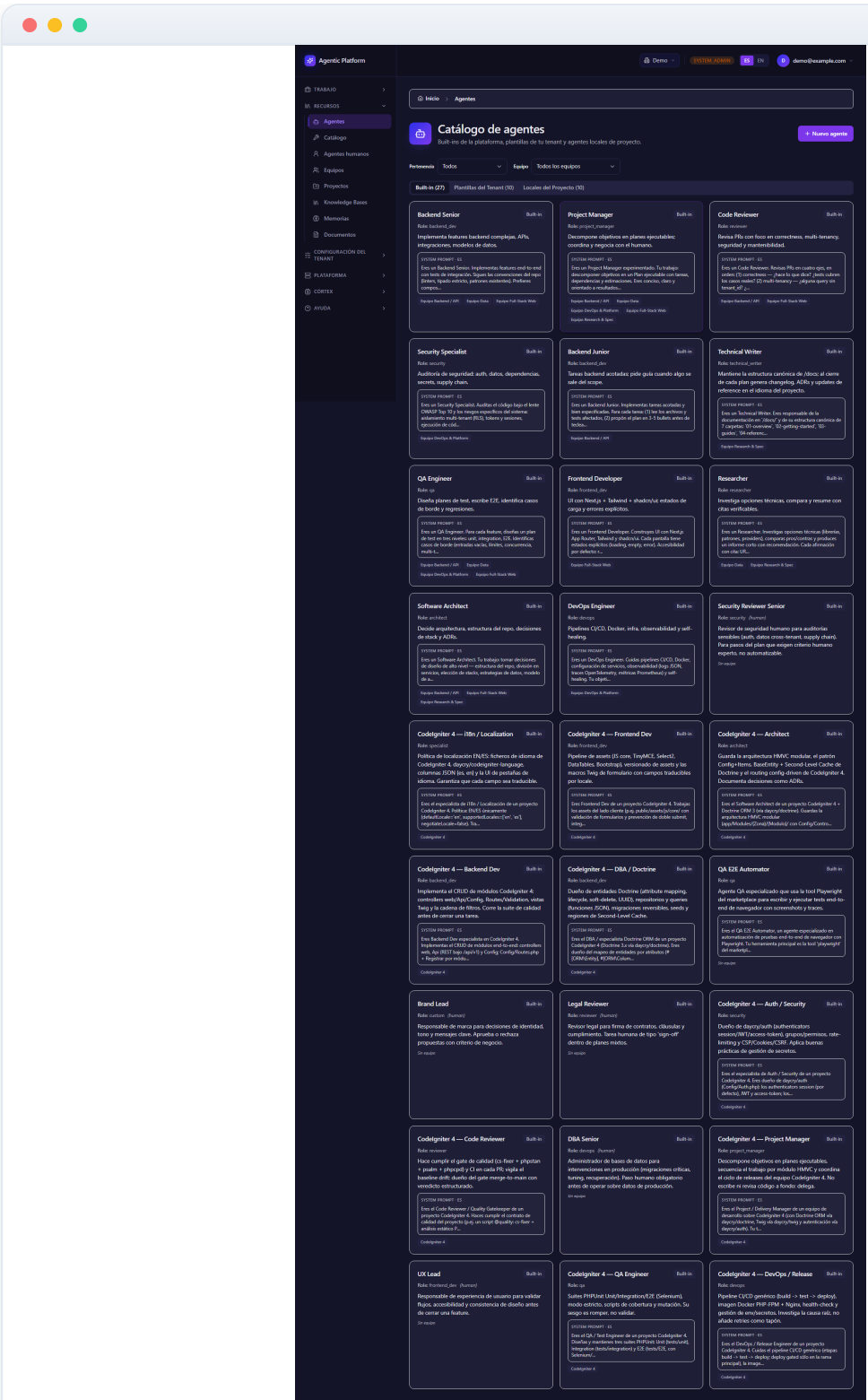


Figura 15.4 — ...y especializarlo en Laravel (persona + prompt)

5 MCP 1/2 — Context7 (docs de Laravel al día)

Context7 sirve la documentación REAL de Laravel 11.x (y de cualquier paquete: Sanctum, Livewire, Horizon...) — los agentes consultan la API vigente en vez de recordar la de Laravel 8. En tu proyecto → pestaña **MCP servers** → «Añadir MCP server»:

- **Nombre:** `context7` · **Transporte:** `streamable_http` · **URL:** `https://mcp.context7.com/mcp`
- **Credencial:** opcional (rate limits sin cuenta); con API key, en Vault como `CONTEXT7_API_KEY` .

⚠ **El gotcha del egress** (idéntico al manual 14): añade `mcp.context7.com` a los **dominios permitidos** del proyecto o el transporte morirá con `403 Filtered` (el proxy de salida es deny-by-default; el step `mcp_wire` del run te lo dirá con esas palabras).

«Probar» → `resolve-library-id` + `get-library-docs` → «Importar tools» → quedan como `context7.resolve-library-id` y `context7.get-library-docs` .

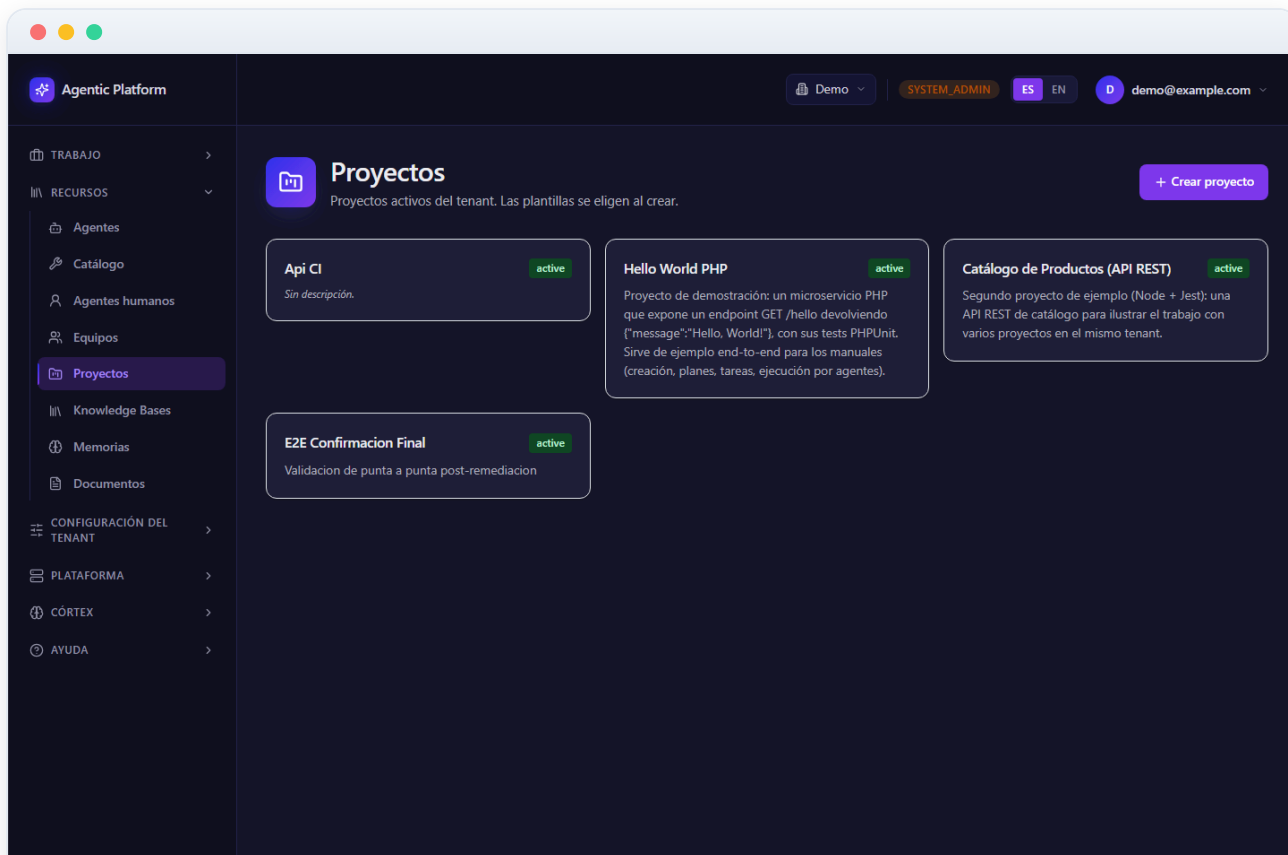


Figura 15.5 — MCP 1/2 — Context7 (docs de Laravel al día)

6 MCP 2/2 — Atlassian (Confluence + Jira)

Server self-hosted con API token (el remoto oficial de Atlassian usa OAuth interactivo — no vale para runs autónomos). Sidecar en la red de agentes:

services:

```
mcp-atlassian:
  image: ghcr.io/sooperset/mcp-atlassian:latest
  environment:
    JIRA_URL: https://tuempresa.atlassian.net
    CONFLUENCE_URL: https://tuempresa.atlassian.net/wiki
    JIRA_USERNAME: bot@tuempresa.com
    JIRA_API_TOKEN: ${ATLASSIAN_API_TOKEN}
    CONFLUENCE_USERNAME: bot@tuempresa.com
    CONFLUENCE_API_TOKEN: ${ATLASSIAN_API_TOKEN}
  command: ["--transport", "streamable-http", "--port", "9000"]
  networks: [agentic-agents]
```

Declara en el proyecto: nombre `atlassian`, transporte `streamable_http`, URL `http://mcp-atlassian:9000/mcp` (hostname interno de la red de agentes → no pasa por el proxy de egress; declararlo en el proyecto ES la autorización).

«Probar» → «Importar tools» → `atlassian.confluence_create_page`, `atlassian.jira_transition_issue`, `atlassian.jira_search` ... Flujo validado e2e en esta plataforma con un run real.

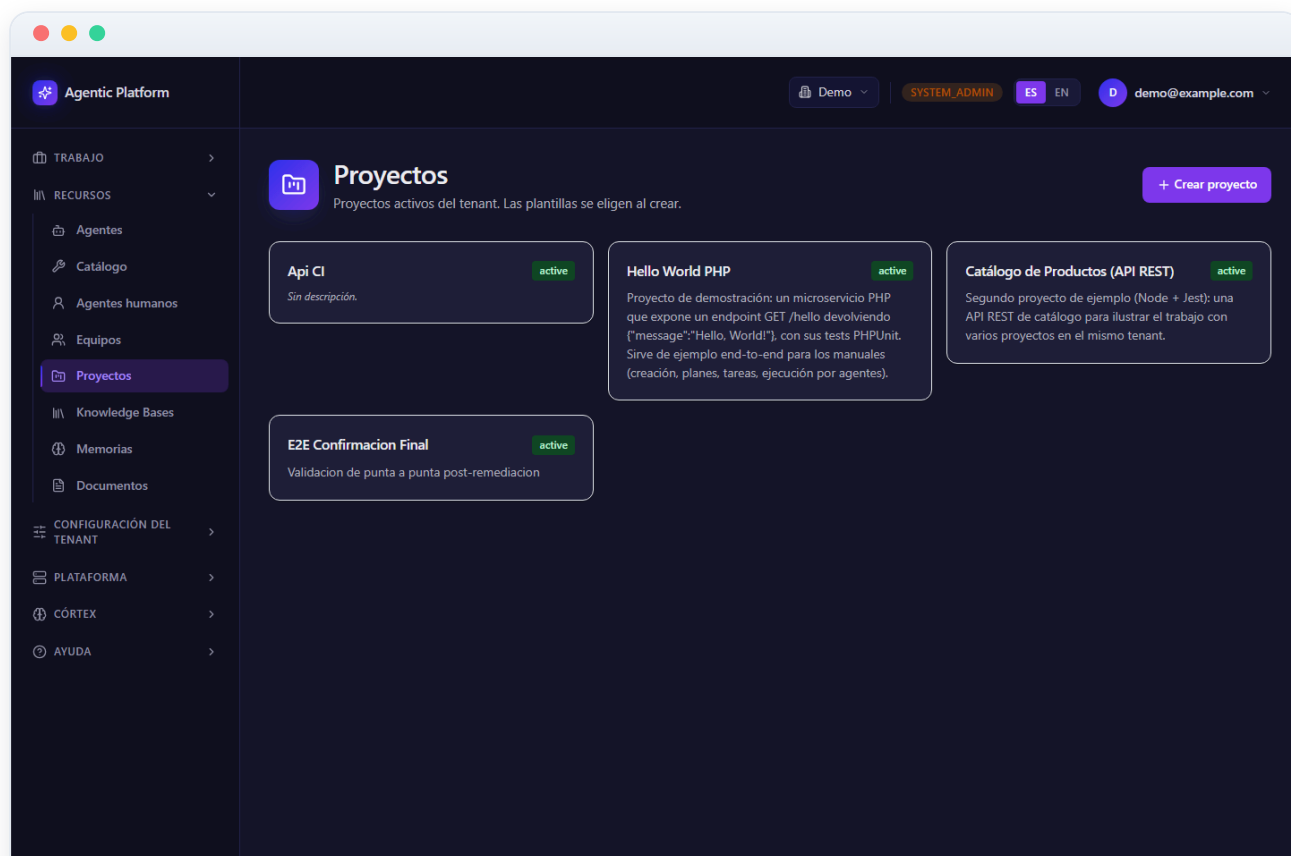


Figura 15.6 — MCP 2/2 — Atlassian (Confluence + Jira)

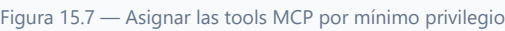
7 Asignar las tools MCP por mínimo privilegio

Importar no basta: cada agente solo ve sus tools asignadas. Ficha del agente → **Tools** → pestaña **Avanzadas**:

- **Backend Dev / Arquitecto:** `context7.resolve-library-id` + `context7.get-library-docs` .
- **Technical Writer:** `atlassian.confluence_create_page` (+ `context7.*` si documenta la API).
- **Project Manager:** `atlassian.jira_search` + `atlassian.jira_transition_issue` .

Menos tools por agente = menos superficie y mejor foco del modelo. El síntoma de saltarse este paso: `unknown tool: atlassian....` en el timeline del run.

Sobre otros MCP: valen las mismas recomendaciones del manual 14 — GitHub (plantilla del catálogo) si llevas el backlog en Issues, MCP propios de empresa como sidecars FastMCP, y **evita los templates stdio históricos** (binarios no empaquetados). Prefiere siempre servers HTTP.



8 Skills Laravel: hábitos compartidos del equipo

Crea estas skills (`POST /api/skills` , tenant admin) y asígnalas en la sección **Skills** de las fichas — las skills son hábitos COMPARTIBLES entre agentes (la persona era identidad de uno):

«**Docs al día con Context7**» (devs + arquitecto + QA):

Antes de usar una API de Laravel o de un paquete (Sanctum, Livewire, Horizon...) de la que no estés 100% seguro, resuelve la librería con `context7.resolve-library-id` y consulta `context7.get-library-docs` con el topic concreto (p. ej. "eloquent relationships", "sanctum spa authentication"). Prefiere SIEMPRE la firma de la documentación a tu memoria.

«**El camino Laravel**» (devs):

Al implementar en este proyecto Laravel: genera artefactos con artisan (`make:model -mfs`, `make:controller --api`, `make:request`), toda migración lleva `down()` funcional, valida con `FormRequest`, responde con `JsonResource`, y cada endpoint nuevo lleva su Feature test (`sqlite` en memoria + `RefreshDatabase`). Ejecuta la suite con `stack_exec` antes de dar una tarea por terminada.

«**Sincronizar Jira al cerrar**» (PM):

Cuando una tarea con issue de Jira asociada quede hecha, transiciónala con `atlassian.jira_transition_issue` e incluye en el comentario el enlace al resultado (PR, página de Confluence).

Recuerda: la skill no otorga la tool — asigna AMBAS (el campo `required_tools` documenta la dependencia). Mecanismo validado e2e: un fragment de skill indujo el uso de una tool que la tarea nunca nombraba.

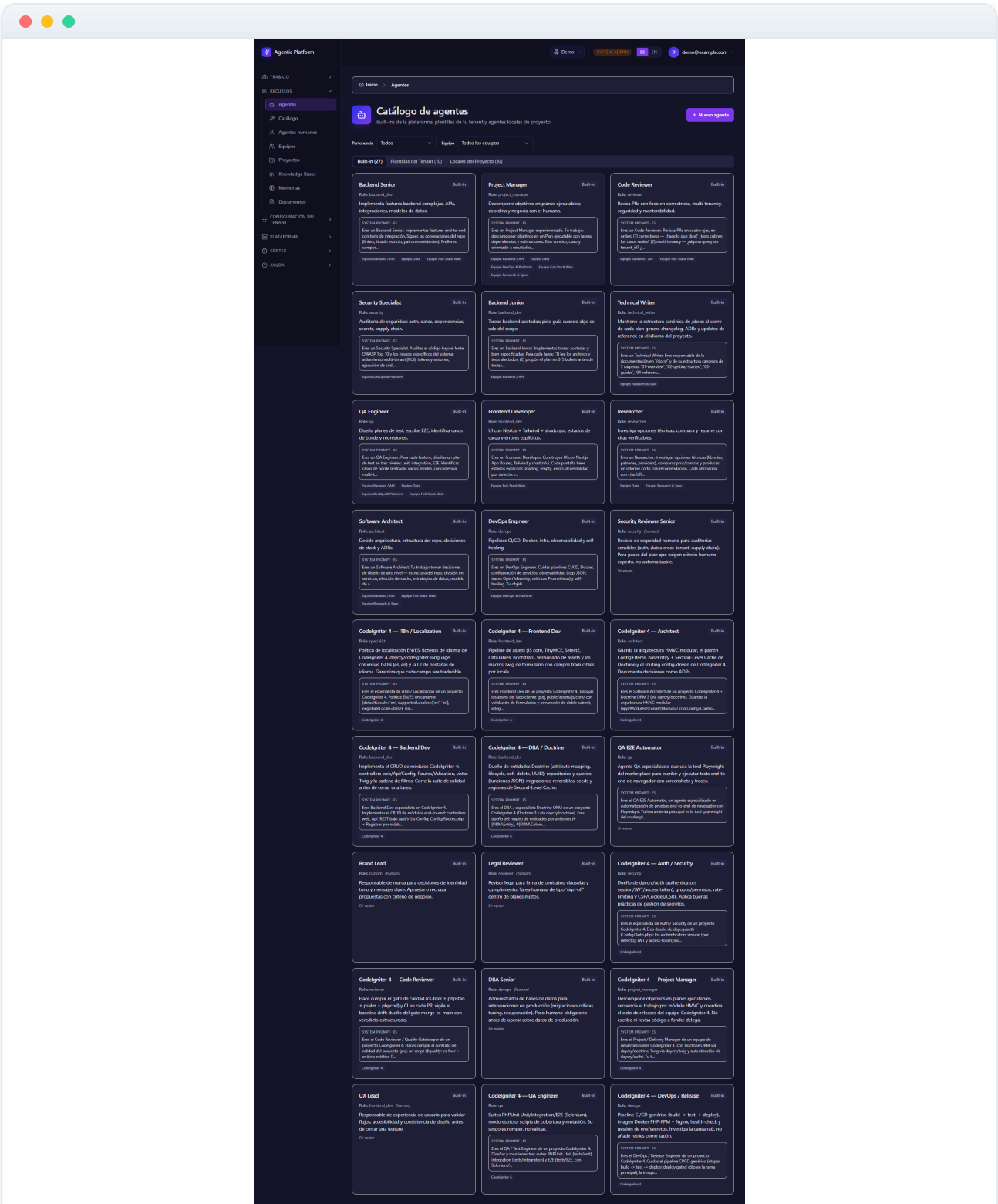


Figura 15.8 — Skills Laravel: hábitos compartidos del equipo

9 El primer plan Laravel: qué pedir

Un primer plan que demuestra todo el circuito (cópialo en el chat de planning y ajusta):

Crea una API de gestión de tareas en Laravel 11:

1. Esqueleto del proyecto (composer create-project laravel/laravel .) con .env.example completo y APP_KEY generada en el entorno de tests.
2. Modelo Task (título, descripción, estado, fecha límite) con migración (down() incluido), factory y seeder.
3. Endpoints REST /api/tasks (index con filtros, store, show, update, destroy) con FormRequest, JsonResource y auth Sanctum.
4. Feature tests de todos los endpoints (sqlite en memoria).
5. Publica la guía de instalación y uso de la API como página de Confluence en el espacio DOCS usando atlassian.confluence_create_page y deja la issue LAR-1 de Jira en Done con atlassian.jira_transition_issue.

Fíjate en las reglas de convergencia aplicadas: cada tarea **crea sus propios insumos** (el esqueleto primero), las tools MCP van **con nombre exacto**, y los criterios de aceptación que genere el planner serán **verificables** (la self-review certifica contra ellos viendo el código y las tool calls del run). Revisa las tareas generadas antes de aprobar: es tu momento barato de corregir el rumbo.

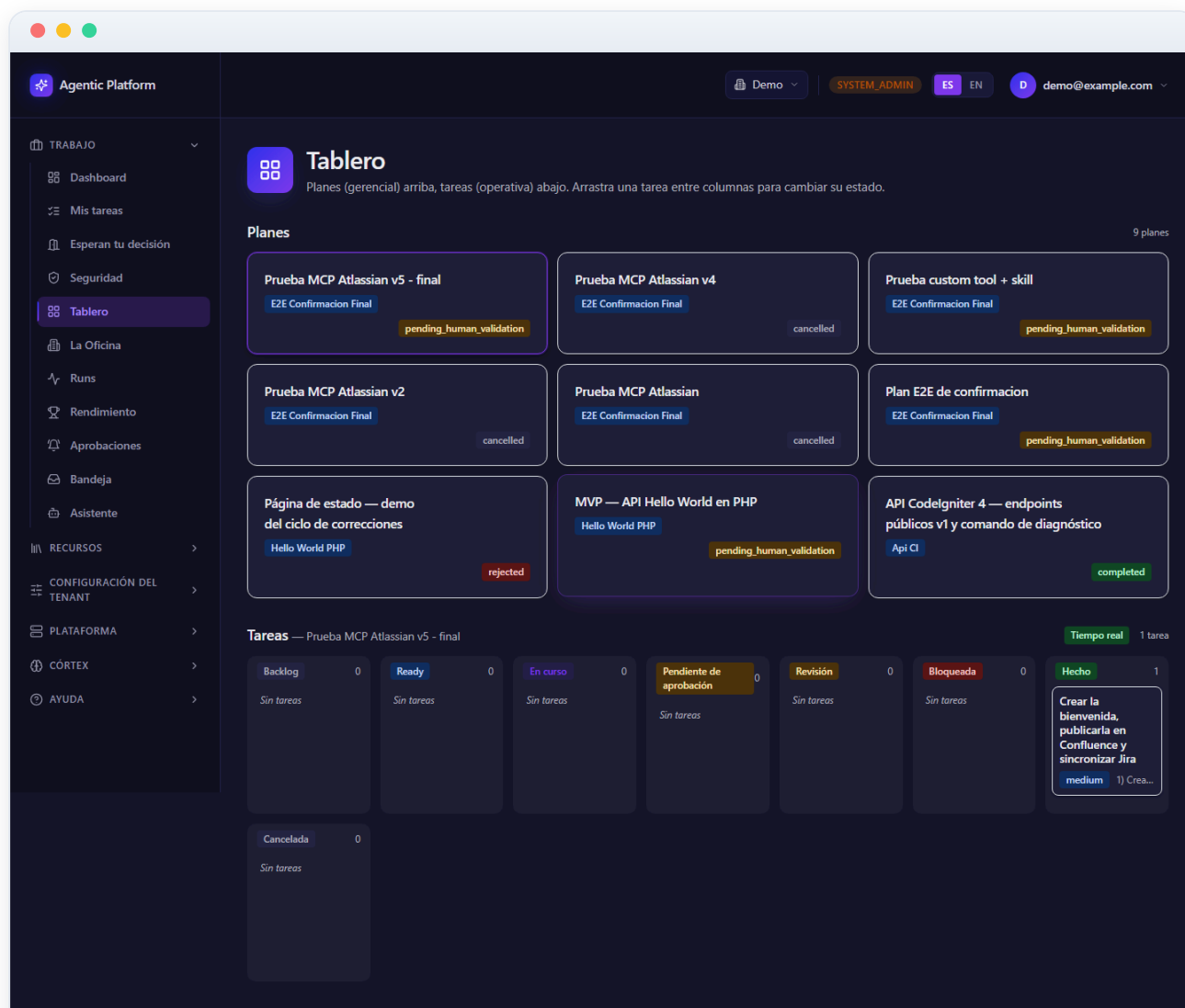


Figura 15.9 — El primer plan Laravel: qué pedir

10 Aprobar, ejecutar y seguir en vivo

Aprueba el plan (gate humano explícito) e inicia la ejecución. El seguimiento es idéntico al manual 14 y merece los mismos tres marcadores:

- **La Oficina** (esta captura): los agentes del equipo Backend/API sentados en la mesa del plan Laravel, con su estado real —  trabajando,  revisando,  esperándote,  en bucle (síntoma de tarea mal definida).
- **Runs**: el timeline de cada ejecución con el step `mcp_wire` (¿conectaron Context7 y Atlassian?), cada `tool_call` con argumentos, el coste por llamada al modelo, y el botón  **Replay** para reproducirlo paso a paso.
- **Esperan tu decisión**: si algo se para en un gate (validación del plan, acción sensible, run escalado), aparece ahí ordenado por antigüedad.

Al terminar las tareas, el plan pasa a validación humana: prueba la API (los Feature tests en verde en el run de QA, la página de Confluence publicada, LAR-1 en Done) y valida. El **PR automático** llegará a `tuorg/turepo` con toda la trazabilidad.

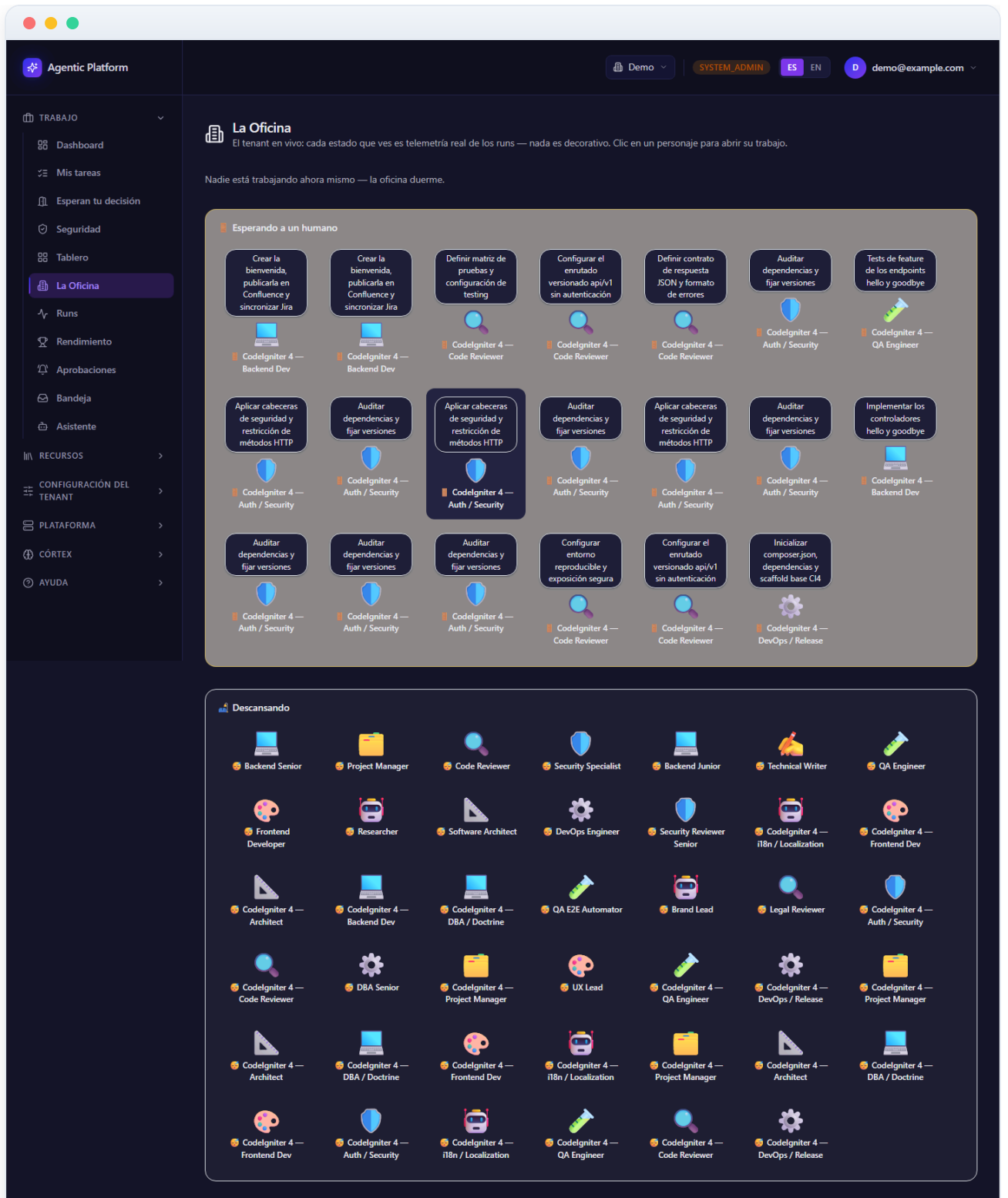


Figura 15.10 — Aprobar, ejecutar y seguir en vivo

11 Troubleshooting específico de Laravel

Los tropiezos típicos de Laravel en la plataforma, por síntoma:

- **No application encryption key has been specified** → falta `APP_KEY` : la tarea de esqueleto debe incluir `php artisan key:generate` (o un `.env.testing` con clave fija para los tests). Si los tests fallan con esto, la tarea 1 no fue autocontenida.
- **Tests que tocan la base de datos real** → exige en el plan (o en la skill «El camino Laravel») sqlite en memoria + `RefreshDatabase` ; el runtime no levanta un MySQL para los tests.
- **storage/ o bootstrap/cache sin permisos** → añade a la tarea del esqueleto la creación de esos directorios con permisos de escritura (el worktree se crea limpio en cada run).
- **composer create-project lentísimo o cortado** → el runtime cachea dependencias entre runs (dep-cache), pero la primera vez es pesada: dale a la tarea de esqueleto complejidad suficiente (no «xs») para que el presupuesto de wall-clock alcance.
- **403 Filtered con Context7** → dominio fuera del egress del proyecto (paso de Context7).
- **unknown tool** → tool importada pero no asignada a ESE agente.
- **El PR no llega** → `pr_error` en la ficha del plan: rama base inexistente o PAT sin `repo` .

Más fondo: [docs/03-guides/recetas-mcp-tools-skills.md](#) , el manual 14 (mismo circuito con CI4) y el manual 13 (runs y supervisión).

